DS3 Data Science for Developing Scholars in Down Syndrome Research

Introduction to using R and RStudio

Matthew Galbraith
Linda Crnic Institute for Down Syndrome

Data Science for Developing Scholars in Down Syndrome Research (DS3) 2026

INCLUDE INvestigation of Co-occurring conditions across the Lifespan to Understand Down syndromE

Linda Crnic Institute for Down Syndrome
UNIVERSITY OF COLORADO ANSCHUTZ MEDICAL CAMPUS

GLOBAL DOWN SYNDROME FOUNDATION®

Links for this session

Course materials

DS3 2026 repository on GitHub

Installation & setup (README on GitHub)

RStudio & Projects

RStudio getting started guide

RStudio projects guide

R4DS scripts & projects walkthrough

DS3 R project template

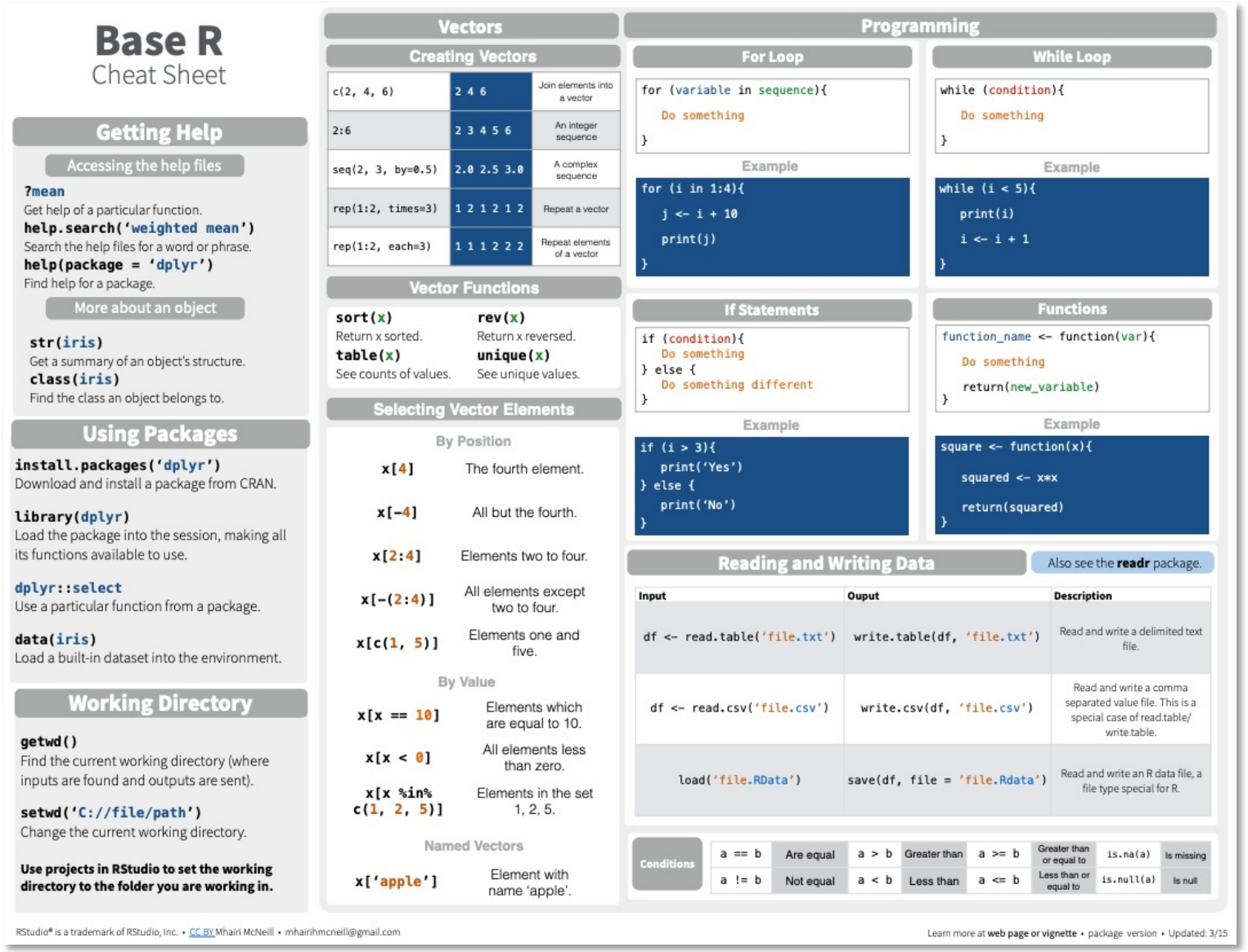
R & data visualization

Tidyverse

https://www.data-to-viz.com/caveats.html

https://r-graph-gallery.com/index.html

R



Base R
Cheat Sheet

Getting Help

Accessing the help files

?mean
Get help of a particular function.
help.search('weighted mean')
Search the help files for a word or phrase.
help(package = 'dplyr')
Find help for a package.

More about an object

str(iris)
Get a summary of an object's structure.
class(iris)
Find the class an object belongs to.

Using Packages

install.packages('dplyr')
Download and install a package from CRAN.

library(dplyr)
Load the package into the session, making all its functions available to use.

dplyr::select
Use a particular function from a package.

data(iris)
Load a built-in dataset into the environment.

Working Directory

getwd()
Find the current working directory (where inputs are found and outputs are sent).

setwd('C://file/path')
Change the current working directory.

Use projects in RStudio to set the working directory to the folder you are working in.

Vectors

Creating Vectors

c(2, 4, 6)	2 4 6	Join elements into a vector
2:6	2 3 4 5 6	An integer sequence
seq(2, 3, by=0.5)	2.0 2.5 3.0	A complex sequence
rep(1:2, times=3)	1 2 1 2 1 2	Repeat a vector
rep(1:2, each=3)	1 1 1 2 2 2	Repeat elements of a vector

Vector Functions

sort(x)
Return x sorted.
rev(x)
Return x reversed.
table(x)
See counts of values.
unique(x)
See unique values.

Selecting Vector Elements

By Position

x[4]	The fourth element.
x[-4]	All but the fourth.
x[2:4]	Elements two to four.
x[-(2:4)]	All elements except two to four.
x[c(1, 5)]	Elements one and five.

By Value

x[x == 10]	Elements which are equal to 10.
x[x < 0]	All elements less than zero.
x[x %in% c(1, 2, 5)]	Elements in the set 1, 2, 5.

Named Vectors

x['apple']	Element with name 'apple'.

Programming

For Loop

```
for (variable in sequence){
   Do something
}
```

Example

```
for (i in 1:4){
   j <- i + 10
   print(j)
}
```

While Loop

```
while (condition){
   Do something
}
```

Example

```
while (i < 5){
   print(i)
   i <- i + 1
}
```

If Statements

```
if (condition){
   Do something
} else {
   Do something different
}
```

Example

```
if (i > 3){
   print('Yes')
} else {
   print('No')
}
```

Functions

```
function_name <- function(var){
   Do something
   return(new_variable)
}
```

Example

```
square <- function(x){
   squared <- x*x
   return(squared)
}
```

Reading and Writing Data

Also see the **readr** package.

Input	Ouput	Description
df <- read.table('file.txt')	write.table(df, 'file.txt')	Read and write a delimited text file.
df <- read.csv('file.csv')	write.csv(df, 'file.csv')	Read and write a comma separated value file. This is a special case of read.table/ write.table.
load('file.RData')	save(df, file = 'file.Rdata')	Read and write an R data file, a file type special for R.

Conditions

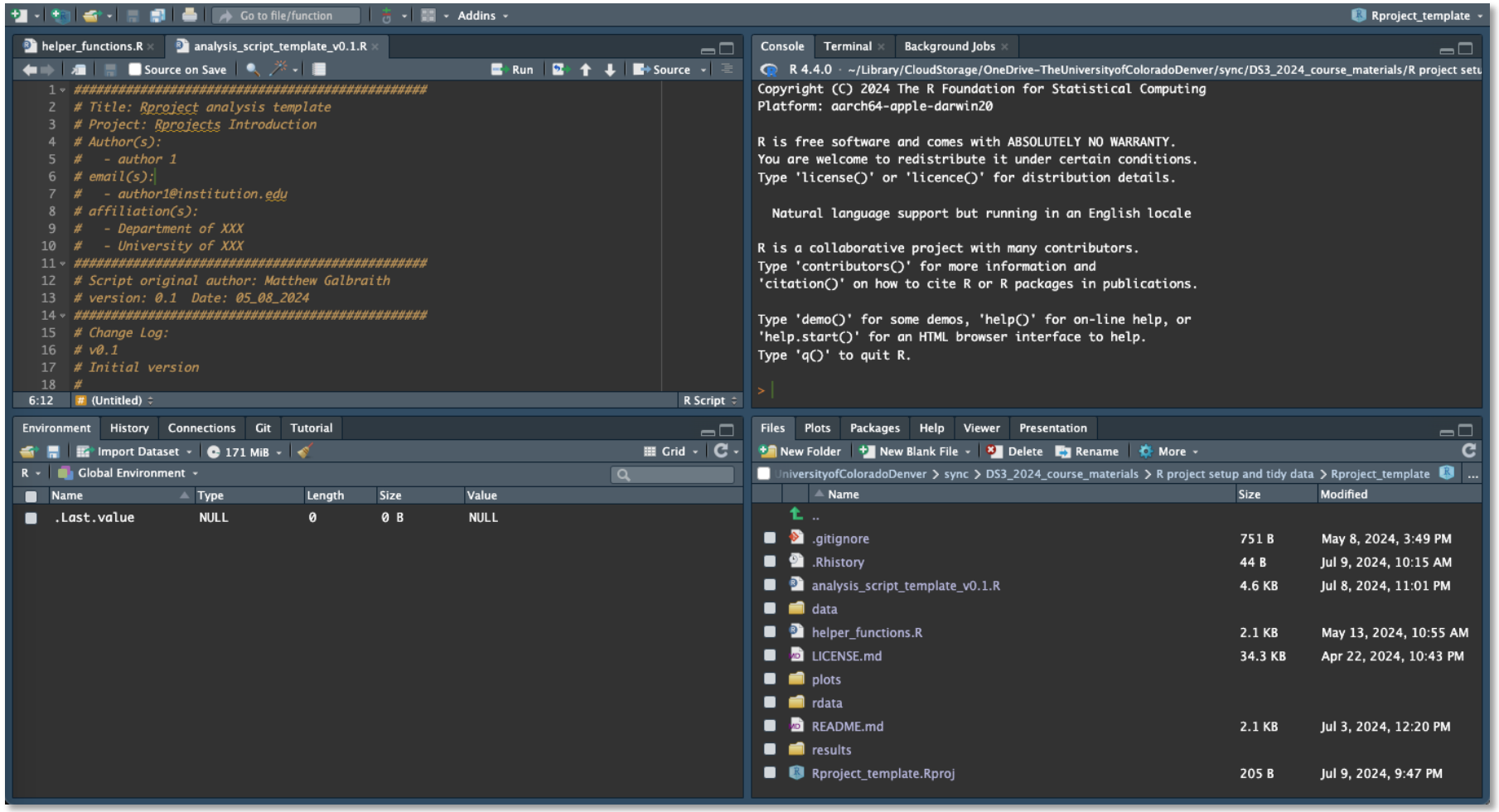
a == b	Are equal	a > b	Greater than	a >= b	Greater than or equal to	is.na(a)	Is missing
a != b	Not equal	a < b	Less than	a <= b	Less than or equal to	is.null(a)	Is null

https://rstudio.github.io/cheatsheets/base-r.pdf

RStudio

Integrated Development Environment (IDE) for R

- Edit + execute R code
- Syntax highlighting, code completion, debugging
- View output, plots, help, environment



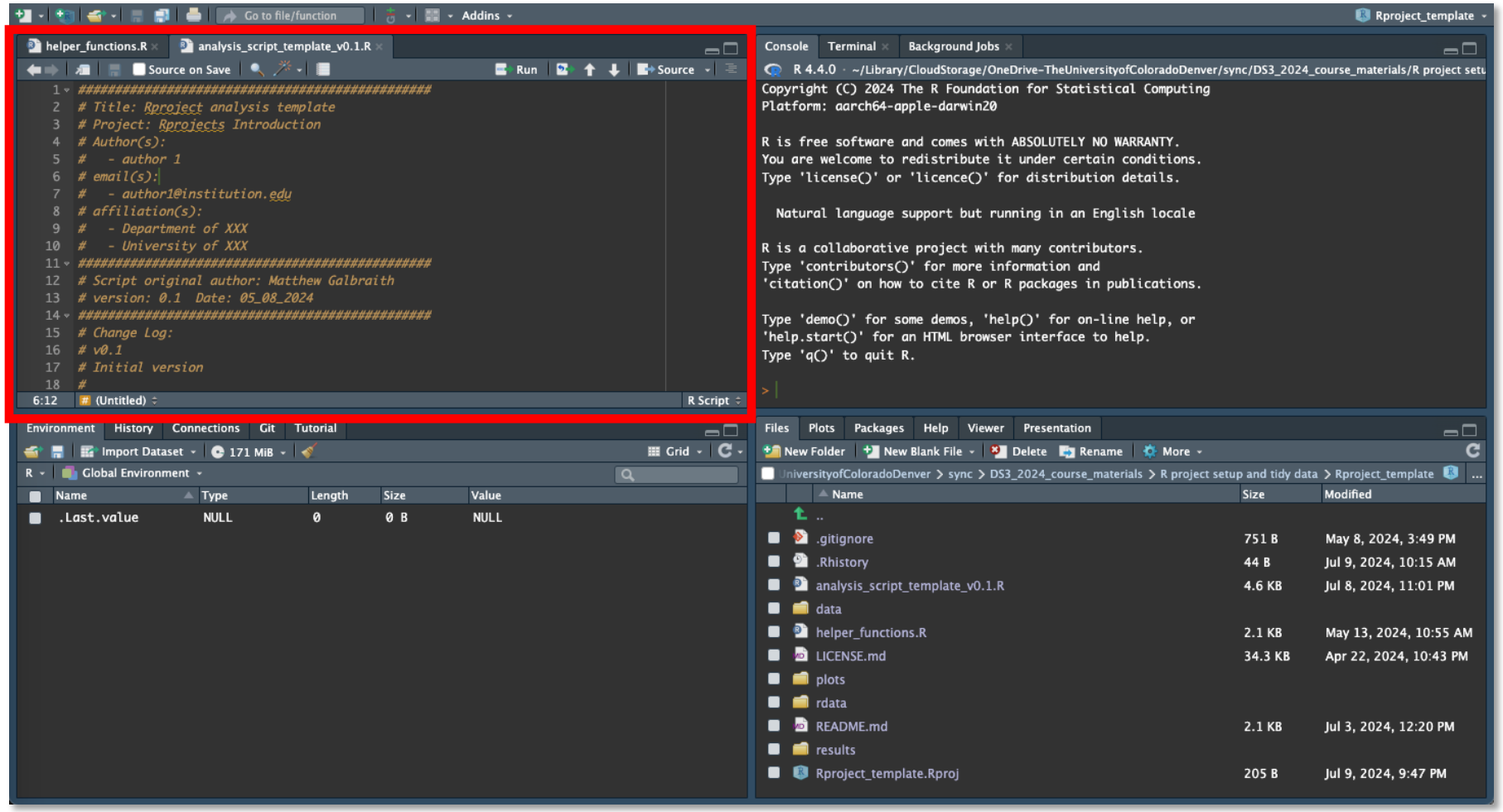
https://posit.co/download/rstudio-desktop/

RStudio

Integrated Development Environment (IDE) for R

- Edit + execute R code
- Syntax highlighting, code completion, debugging
- View output, plots, help, environment

Source pane

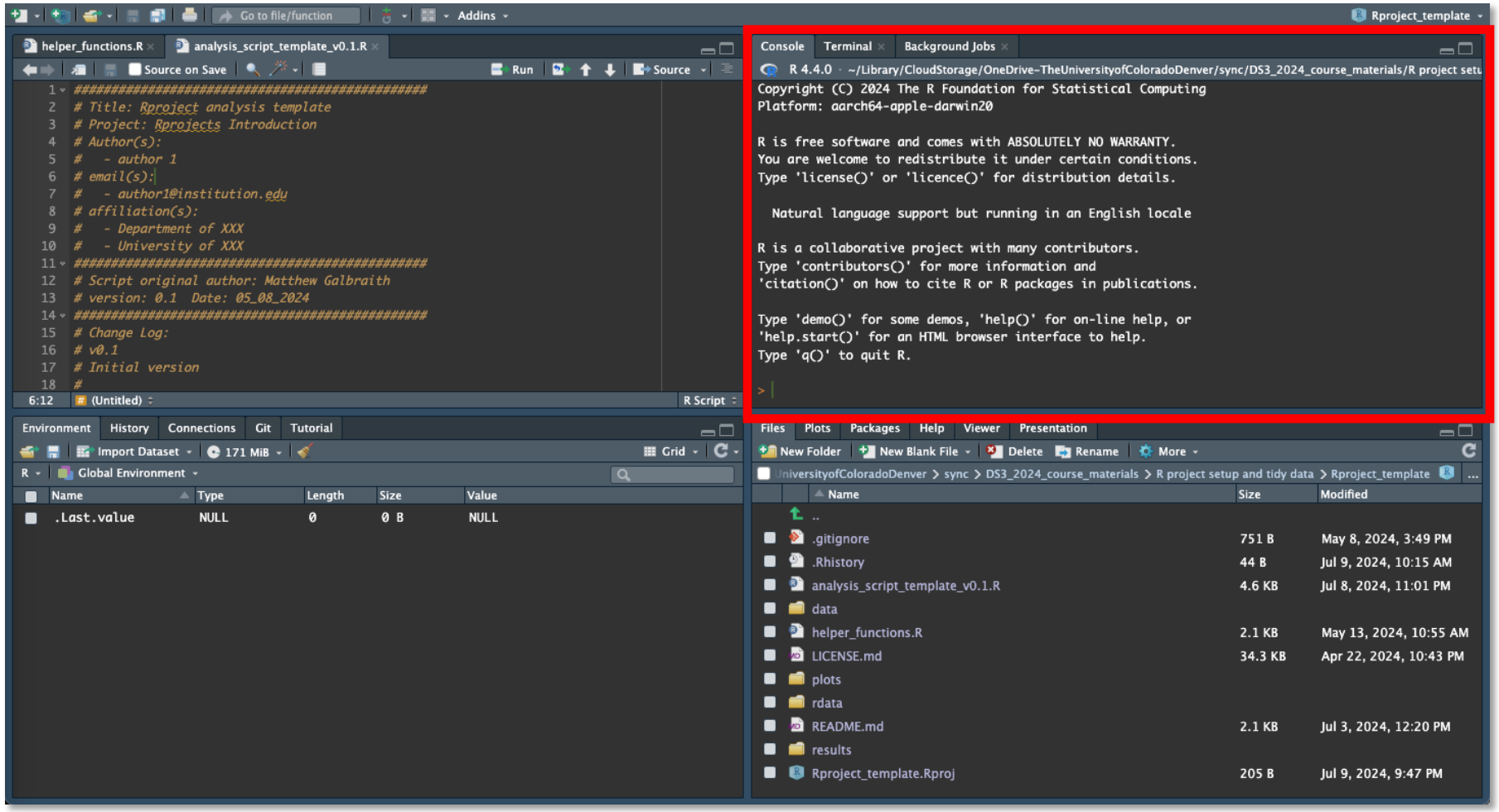


Panes can be rearranged and customized

RStudio

Integrated Development Environment (IDE) for R

- Edit + execute R code
- Syntax highlighting, code completion, debugging
- View output, plots, help, environment



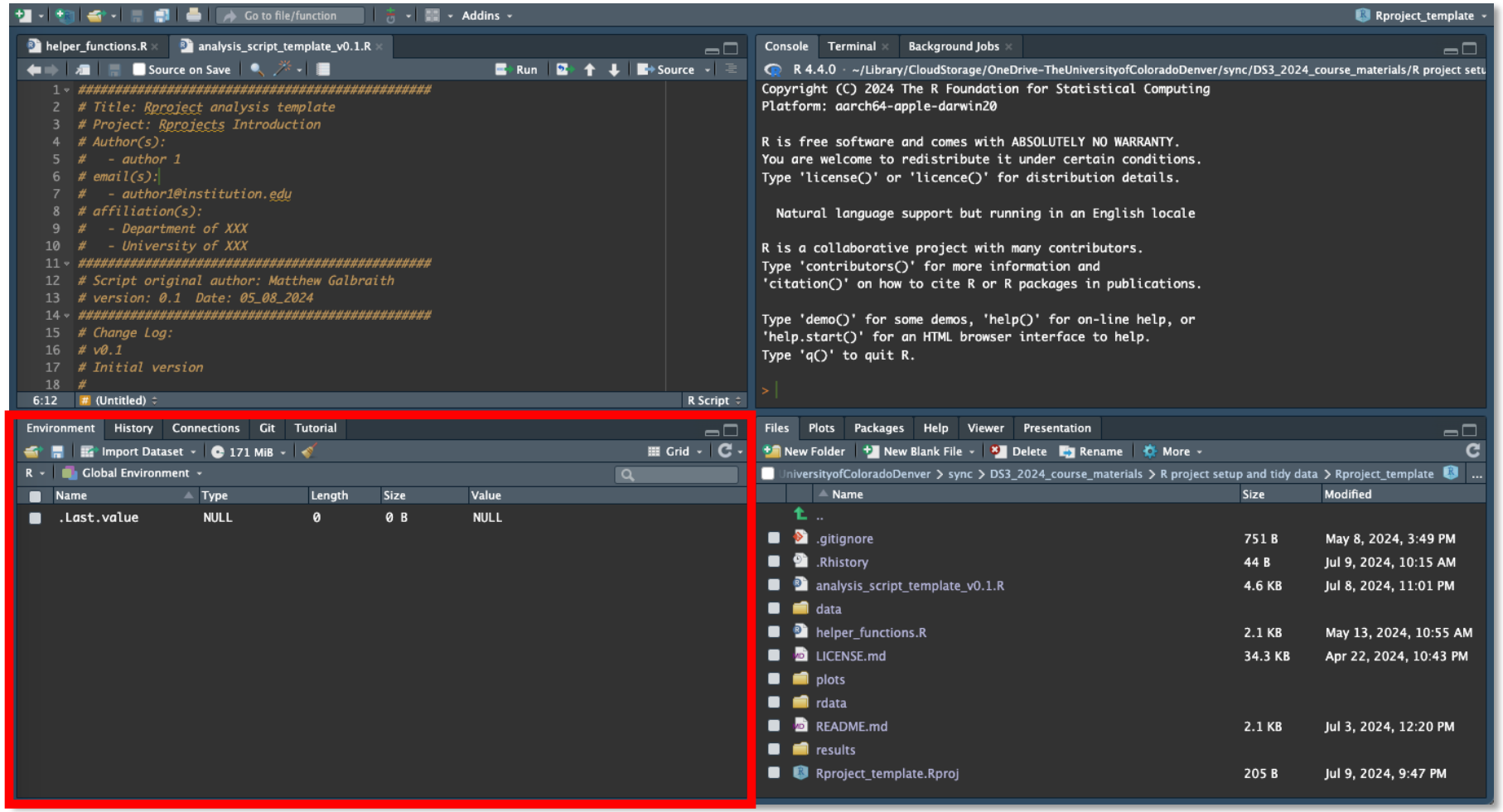
Console pane

RStudio

Integrated Development Environment (IDE) for R

- Edit + execute R code
- Syntax highlighting, code completion, debugging
- View output, plots, help, environment

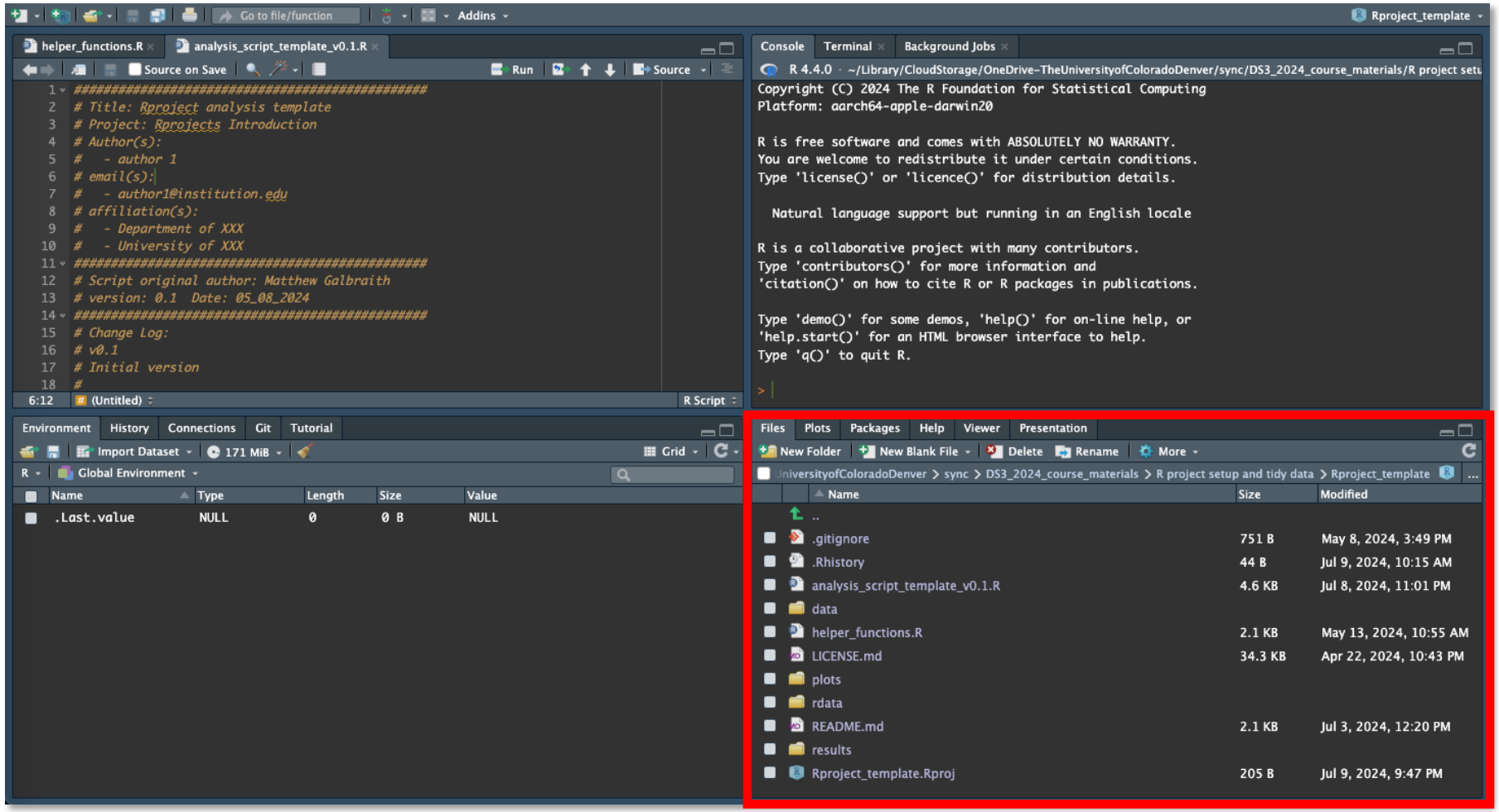
Environment etc pane



RStudio

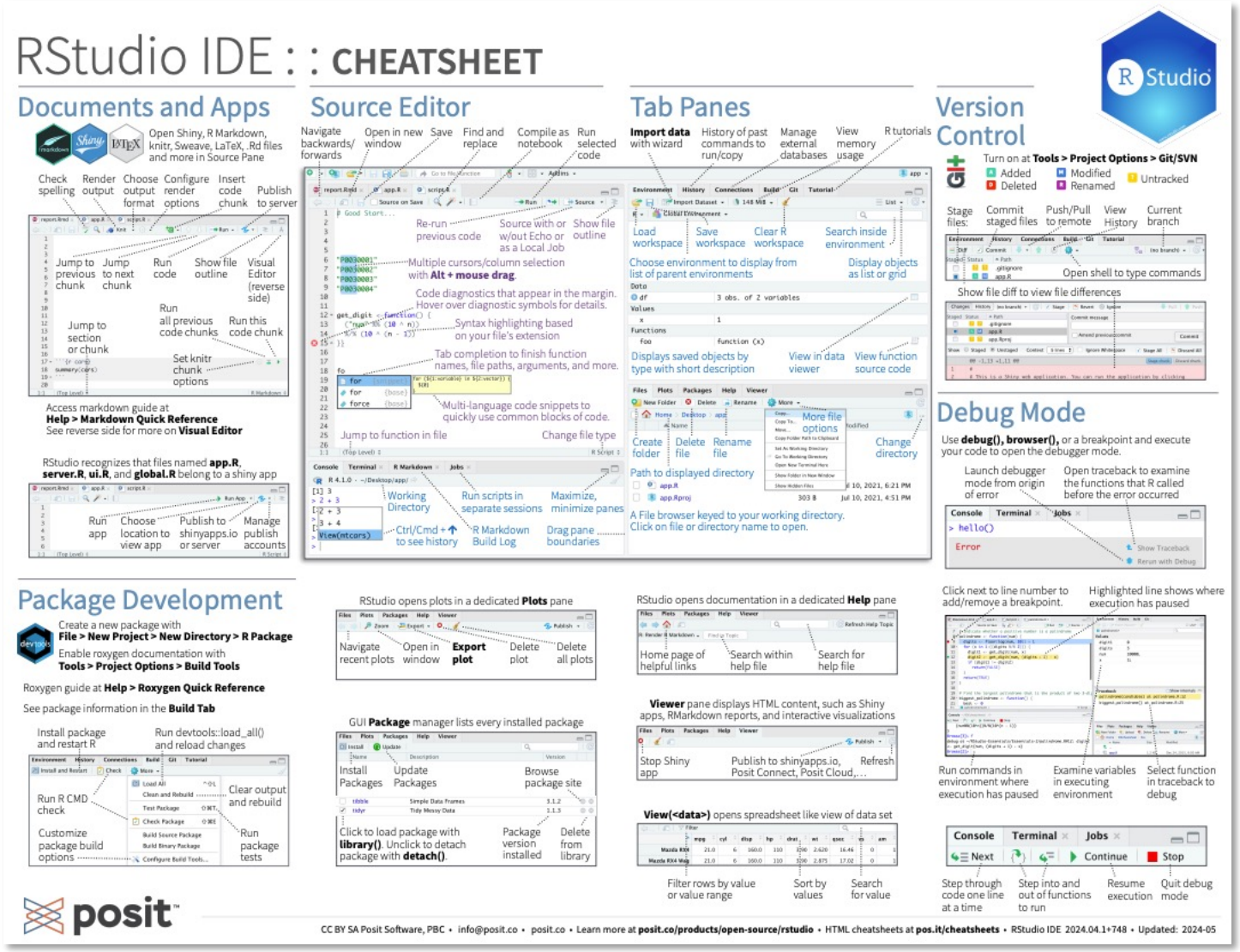
Integrated Development Environment (IDE) for R

- Edit + execute R code
- Syntax highlighting, code completion, debugging
- View output, plots, help, environment



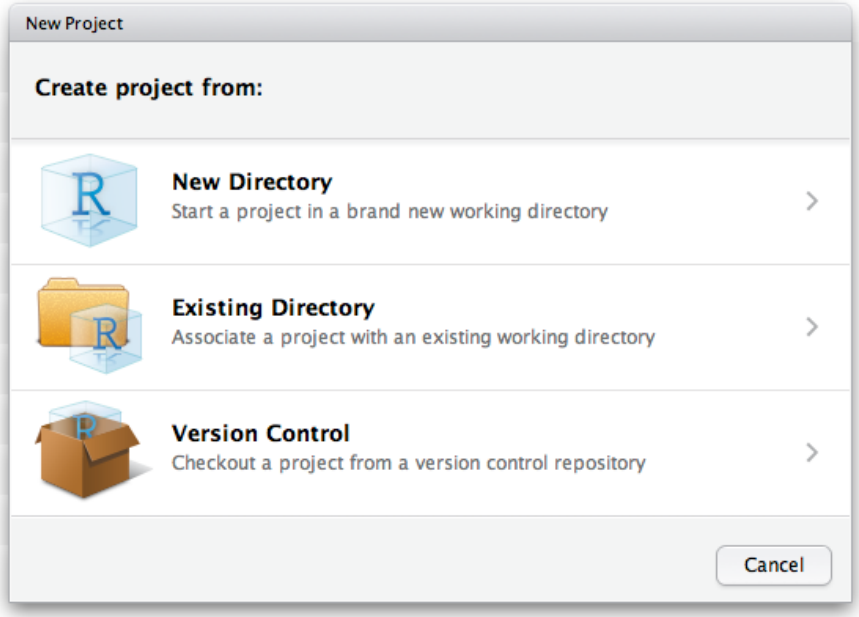
Files, Plots, Help etc pane

RStudio cheatsheet



https://rstudio.github.io/cheatsheets/html/rstudio-ide.html

RStudio Projects



- Open existing projects via .Rproj file
- Automatically sets your working directory
- Self-contained set of directories, scripts, and data files (very important for multiple projects)

Organizing your Rstudio Projects

- Only **/data** and R scripts are required - everything else can be recreated (incl. earlier versions)
- Treat **/data** directory as read-only
- Analysis outputs go to **/results** or **/plots** (with version info)
- R workspace and large intermediate files stored in **/rdata**
- Additional directories added as needed, eg /Archive
- Compatible with manual or other version control

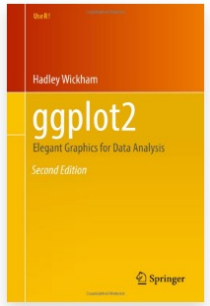
Project_directory
- **/data**
- /results
- /plots
- /rdata
- **analysis_script.R**
- **helper_functions.R**
- project.Rproj

RStudio projects guide
R4DS scripts & projects walkthrough
DS3 R Project template

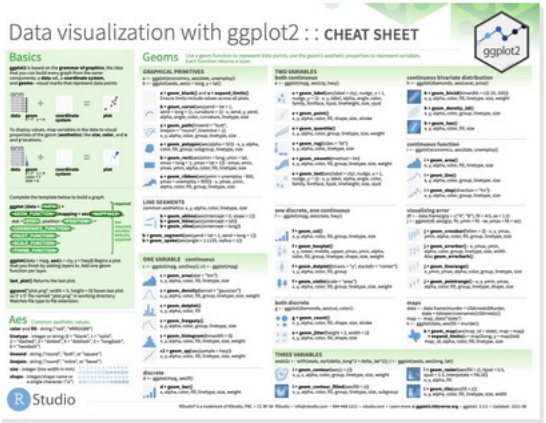
Into the Tidyverse: Visualization using ggplot2

Publication-quality data visualization

- Implements a “grammar of graphics”
- Start by defining the data to be plotted (“aesthetics”):
  `ggplot(aes(x, y, color, fill, shape, alpha, linetype))`
- Then add layers (“geoms”) to specify how data is plotted, eg:
  `+ geom_point()`
- Add additional geom layers, eg:
  `+ geom_boxplot()`
- Can split into separate plots, eg male vs. female, by “faceting”:
  `+ facet_wrap(~ Sex)`
- Finally add title and modify theme:
  `+ labs(title = “Plot title”, subtitle = “plot details”`
  `+ theme(aspect.ratio = 1)`



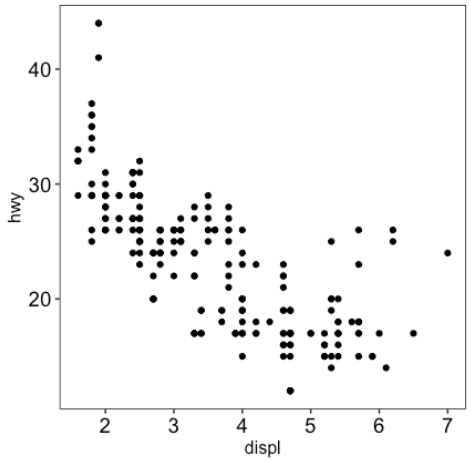
https://ggplot2-book.org/



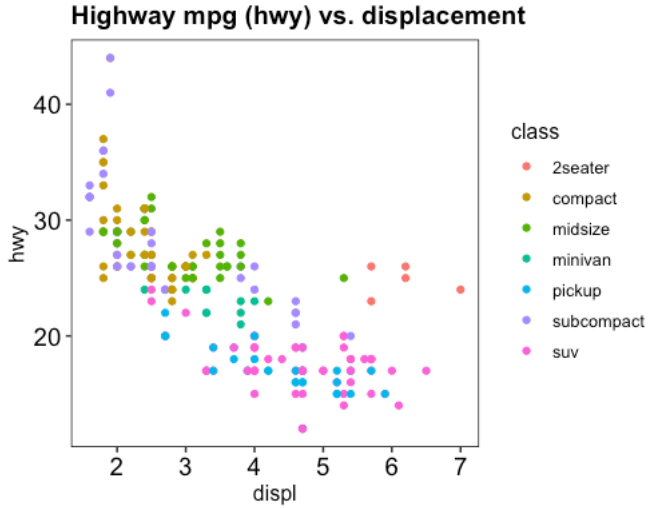
https://rstudio.github.io/cheatsheets/html/data-visualization.html
https://www.data-to-viz.com/caveats.html
https://r-graph-gallery.com/index.html

Visualization using ggplot2


```
mpg %>%
    ggplot(aes(displ, hwy)) +
    geom_point()
```




```
mpg %>%
    ggplot(aes(displ, hwy, color = class)) +
    geom_point() +
    theme(aspect.ratio = 1) +
    labs(title = "Highway mpg (hwy) vs. displacement")
```



Git and GitHub

Version control system

- Tracks changes to code in a repository (and who made them).
- Revert to earlier versions if something breaks.
- Share and collaborate by synchronizing with remote repositories.

Cloud platform for hosting and sharing Git repositories

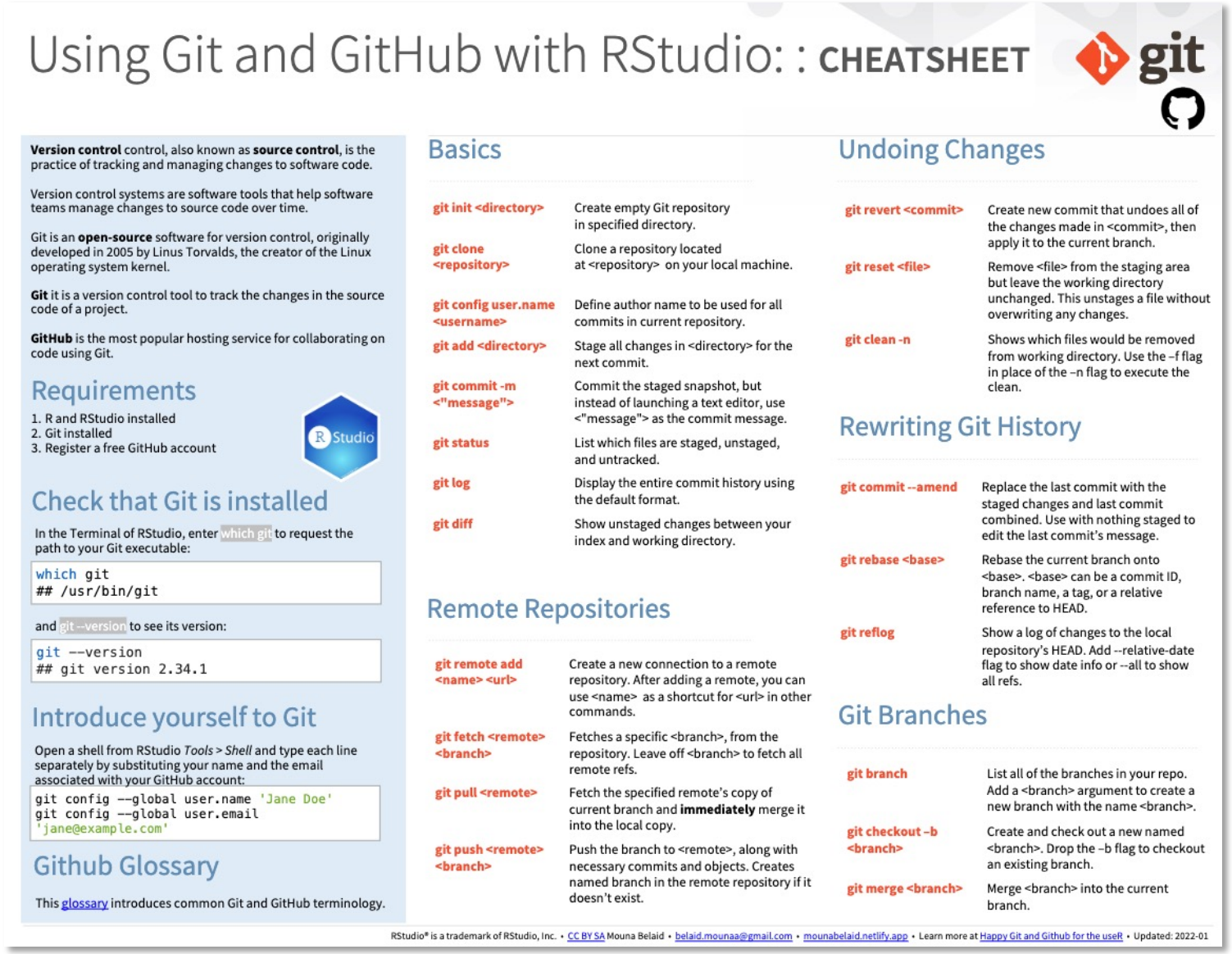
- Works with git version control
- Repository storage: provides a safe, central location to save projects so you can access them from anywhere.
- Enables collaboration across users and teams.

Key Git Concepts

- Repository: A folder where Git tracks your project and its history.
- Clone: Make a copy of a remote repository on your computer.
- Stage: Tell Git which changes you want to save next.
- Commit: Save a snapshot of your staged changes.
- Branch: Work on different versions or features at the same time.
- Merge: Combine changes from different branches.
- Pull: Get the latest changes from a remote repository.
- Push: Send your changes to a remote repository.

https://git-scm.com/
https://github.com/

Git and Github



Using Git and GitHub with RStudio: : CHEATSHEET

Version control control, also known as **source control**, is the practice of tracking and managing changes to software code.

Version control systems are software tools that help software teams manage changes to source code over time.

Git is an **open-source** software for version control, originally developed in 2005 by Linus Torvalds, the creator of the Linux operating system kernel.

Git it is a version control tool to track the changes in the source code of a project.

GitHub is the most popular hosting service for collaborating on code using Git.

Requirements

1. R and RStudio installed
2. Git installed
3. Register a free GitHub account

Check that Git is installed

In the Terminal of RStudio, enter `which git` to request the path to your Git executable:

```
which git
## /usr/bin/git
```

and `git --version` to see its version:

```
git --version
## git version 2.34.1
```

Introduce yourself to Git

Open a shell from RStudio *Tools > Shell* and type each line separately by substituting your name and the email associated with your GitHub account:

```
git config --global user.name 'Jane Doe'
git config --global user.email
'jane@example.com'
```

Github Glossary

This glossary introduces common Git and GitHub terminology.

Basics

git init <directory>	Create empty Git repository in specified directory.
git clone <repository>	Clone a repository located at <repository> on your local machine.
git config user.name <username>	Define author name to be used for all commits in current repository.
git add <directory>	Stage all changes in <directory> for the next commit.
git commit -m <"message">	Commit the staged snapshot, but instead of launching a text editor, use <"message"> as the commit message.
git status	List which files are staged, unstaged, and untracked.
git log	Display the entire commit history using the default format.
git diff	Show unstaged changes between your index and working directory.

Remote Repositories

git remote add <name> <url>	Create a new connection to a remote repository. After adding a remote, you can use <name> as a shortcut for <url> in other commands.
git fetch <remote> <branch>	Fetches a specific <branch>, from the repository. Leave off <branch> to fetch all remote refs.
git pull <remote>	Fetch the specified remote's copy of current branch and **immediately** merge it into the local copy.
git push <remote> <branch>	Push the branch to <remote>, along with necessary commits and objects. Creates named branch in the remote repository if it doesn't exist.

Undoing Changes

git revert <commit>	Create new commit that undoes all of the changes made in <commit>, then apply it to the current branch.
git reset <file>	Remove <file> from the staging area but leave the working directory unchanged. This unstages a file without overwriting any changes.
git clean -n	Shows which files would be removed from working directory. Use the –f flag in place of the –n flag to execute the clean.

Rewriting Git History

git commit --amend	Replace the last commit with the staged changes and last commit combined. Use with nothing staged to edit the last commit's message.
git rebase <base>	Rebase the current branch onto <base>. <base> can be a commit ID, branch name, a tag, or a relative reference to HEAD.
git reflog	Show a log of changes to the local repository's HEAD. Add --relative-date flag to show date info or --all to show all refs.

Git Branches

git branch	List all of the branches in your repo. Add a <branch> argument to create a new branch with the name <branch>.
git checkout –b <branch>	Create and check out a new named <branch>. Drop the –b flag to checkout an existing branch.
git merge <branch>	Merge <branch> into the current branch.

https://rstudio.github.io/cheatsheets/git-github.pdf

How we will use Git and GitHub

GitHub repository: https://github.com/DS3-Course/DS3_2026

Simple workflow for accessing and updating course materials for DS3 afternoon sessions

- **Clone** the course repository once at the beginning:
  `git clone https://github.com/DS3-Course/DS3_2026.git`
- **Pull** updates each day to access new content:
  `git pull`
- ***Alternative:*** *direct download from github*
- **Open** the appropriate `.Rproj` file for each lesson in RStudio.
- **Follow** instructions in README.md files and R scripts.